CHAPTER 15

Design YouTube

Upload, transcode and stream video globally with low buffering and optimised cost.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a video-sharing platform (YouTube/Netflix style): users **upload videos** and **watch/stream videos** on web, mobile and TV. Focus on the upload pipeline (transcoding) and the streaming path (CDN, adaptive bitrate).

Why interviewers ask it: Tests handling of very large blobs, async pipelines (DAG), CDN economics, and streaming protocols.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Important features?	Upload video, watch video.
Clients?	Mobile apps, web, smart TV.
DAU?	5 M.
Average daily watch time?	30 minutes.
International users? Formats?	Yes; most common video formats and resolutions.
Encryption? Max file size?	Yes; max 1 GB.
Use cloud?	Yes (AWS/GCP/Azure) - build on managed services.

3. Requirements

Functional

- Upload videos (up to 1 GB).
- Stream/watch videos with adaptive quality.
- Videos playable on many devices and bandwidths.

Non-functional

- Fast, smooth playback (low startup and buffering).
- High availability, scalability, reliability.
- Low infrastructure cost.

4. Back-of-the-envelope estimation

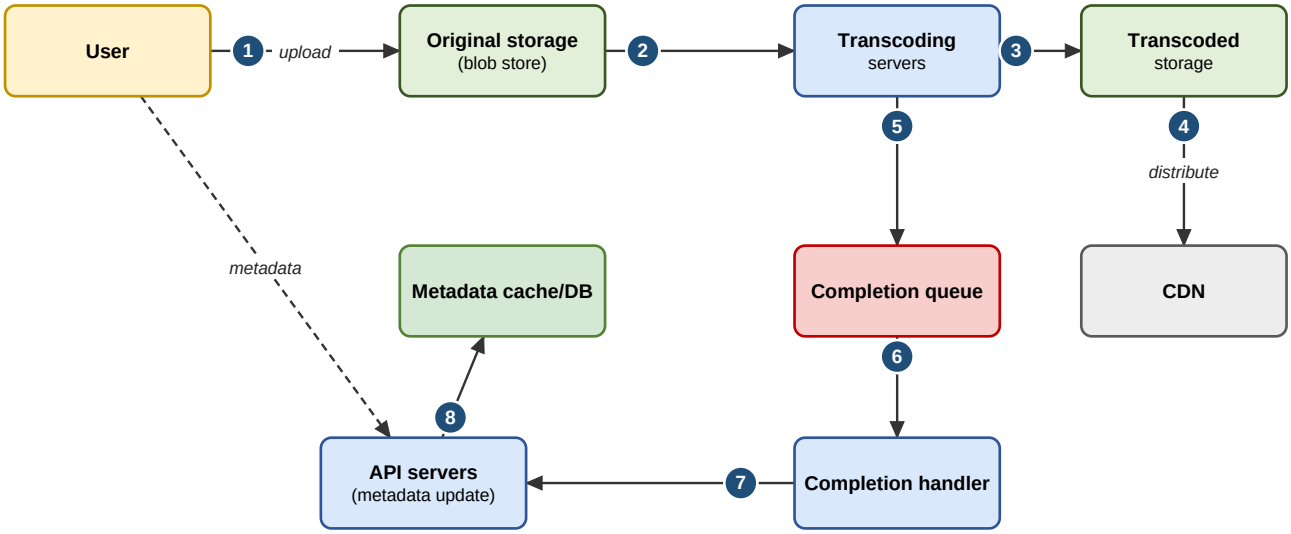
- 5 M DAU x 5 videos/day = 25 M video views/day. If 10% upload 1 video/day: 0.5 M uploads/day x average 300 MB = **150 TB/day** of raw storage (before transcoded copies).
- CDN cost: 5 M x 5 videos x 0.3 GB = 7.5 PB/day egress. At $0.02/GB = about **$150 K/day** -> CDN cost dominates; serve popular videos from CDN, long-tail from your own servers.

5. API / data model

```
POST /videos/upload (resumable chunk upload) -> videoId
GET /videos/{id}/manifest  -> HLS/DASH manifest URL
GET <cdn>/<segment>.ts|.m4s
```

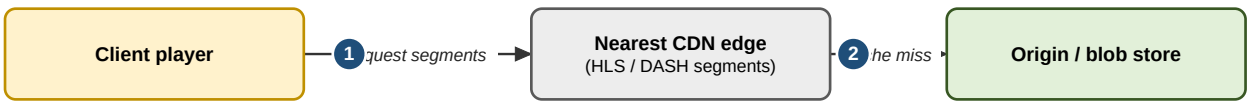
6. High-level design

Video upload flow



Step	What happens
1	The client uploads the video in **parallel, resumable chunks** (using a pre-signed URL) to **original storage** (blob store).
2	Uploading triggers **transcoding**: a preprocessor splits the video into GOP chunks and builds a **DAG** of tasks (encode per bitrate, audio, thumbnail, watermark).
3	Workers encode in parallel; encoded segments and manifests go to **transcoded storage**.
4	Encoded segments are distributed to the **CDN** (popular videos at the edge).
5	When done, the transcoder publishes a completion event to a **queue**.
6	The completion handler consumes the event.
7	It tells the API servers the video is ready.
8	API servers update the **metadata DB/cache** (title, status, URLs). Meanwhile `user -> api` already stored the metadata during upload.

Streaming flow



Step	What happens
1	The player requests a manifest and then small segments (HLS/DASH) from the **nearest CDN edge**, choosing a bitrate from measured bandwidth (**adaptive bitrate**).
2	On a cache miss the edge fetches the segment from the origin/blob store and caches it for later viewers.

7. Deep dive

Video transcoding

- Raw video is huge and many devices need other formats. Encode into **multiple bitrates/resolutions/codecs** (e.g. H.264, VP9, AV1) so players can adapt to bandwidth (adaptive bitrate streaming).
- Container (.mp4, .mov) + codec (H.264...). Output formats compatible with HLS (Apple) and MPEG-DASH.

Transcoding as a DAG pipeline

- Different creators need different steps (watermark, thumbnails, audio-only). Model it as a **Directed Acyclic Graph**: nodes are tasks (video encoding, audio, thumbnail, watermark), edges are dependencies. Components: **preprocessor** (split video into GOP chunks, build DAG), **DAG scheduler**, **resource manager** (task queue, worker queue, running queue), **task workers**, **temporary storage**, **encoded output**.
- Splitting into independent GOP/segment chunks lets many workers encode in parallel -> low latency.

Speeding up uploads

- **Parallel/resumable chunk upload:** client splits the file by GOP alignment and uploads chunks in parallel; failed chunks retry only themselves.
- **Upload centres by geography** (multiple data centers or presigned URLs to the nearest object store) lower latency.
- Pipeline decoupled with **message queues** so each stage runs at its own pace.

Streaming protocols and playback

- Streaming = continuously receiving small segments, not downloading the whole file. **HLS** and **MPEG-DASH** with **adaptive bitrate** based on measured bandwidth.
- Videos served from the CDN edge closest to the viewer.

CDN strategy and cost

- Popular videos (short head) from CDN edge; infrequent ones (long tail) from origin/regional servers to save cost; encode fewer variants for rarely watched videos; prefer ISPs and regional peering where possible. Encode on demand for rare cases.

Safety, security and copyright

- **Pre-signed URLs** for uploads (only authorised clients); **DRM** (Apple FairPlay, Google Widevine, Microsoft PlayReady) and AES encryption; **content moderation** and **copyright detection** pipelines (fingerprinting).

Metadata and storage

- Metadata DB (sharded/replicated MySQL or similar) + cache for titles, tags, view counts; blob store (S3-like) for video; thumbnails via CDN.

Error handling

- Recoverable errors (transcode failed): retry. Non-recoverable (malformed video): stop and return a code. Upload errors: retry chunks; playback errors: switch bitrate/CDN; DB/queue failures: failover/replicas.

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Build vs buy	Build own CDN/storage / use cloud + commercial CDN	Use cloud services unless you are at Google/Netflix scale.
CDN usage	All videos on CDN / only hot videos	Hot videos on CDN, long tail from origin to control cost.
Transcoding	Eager (all formats) / lazy (on demand)	Eager for new/popular; lazy for cold content.
Protocol	HLS / DASH	Support both; HLS needed for iOS.

9. Scaling, reliability, failure modes

- Stateless API servers; metadata DB sharding/replication; blob storage practically unlimited.
- Autoscale transcoding workers from queue depth; use spot instances for non-urgent jobs.
- Multi-region origins; failover between CDNs.

10. Likely follow-up questions

- How do you reduce time-to-first-frame?
- How would you build recommendations/feed?
- How would you support live streaming? (low-latency protocols, smaller segments, ingest servers)
- How would you reduce storage cost for old videos?

11. Common mistakes

- Forgetting transcoding and adaptive bitrate.
- Serving everything from a CDN without a cost discussion.
- Synchronous upload -> encode -> publish path with no queue.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"The two core features are upload video and watch video, on web, mobile and TV, with 5 M DAU, 30 minutes average watch time, global users, files up to 1 GB, and cloud services allowed. Storage is large, about 150 TB a day raw, but CDN egress dominates cost, which will shape the design."
2. High-level design	10 min	"The upload flow: chunked upload to original storage, a transcoding pipeline produces many bitrates and formats, results go to transcoded storage and the CDN, and a completion event updates the metadata DB. The streaming flow: the player fetches HLS or DASH segments from the nearest CDN edge."
3. Deep dive	25 min	"For transcoding I model a DAG of tasks and split videos into GOP chunks so workers encode in parallel, with queues between stages. I speed up uploads with parallel resumable chunks and regional upload endpoints. For cost I put popular videos on the CDN and serve the long tail from origin. For safety: pre-signed URLs, DRM or AES encryption, and content moderation. Errors: retry recoverable ones, stop on malformed video."
4. Wrap up	4 min	"I'd mention live streaming, recommendations, reducing time-to-first-frame, and tiering old videos to cheaper storage. I'd monitor transcoding backlog, CDN hit rate and buffering ratio."

13. 60-second recap / practice checklist

- Upload: chunked/resumable -> blob store -> transcoding DAG -> CDN + metadata update.
- Stream: HLS/DASH segments from nearest CDN with adaptive bitrate.
- Cost control (CDN long tail), security (signed URLs, DRM), failure handling.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• RFC 8216: HTTP Live Streaming (HLS)
• Netflix Tech Blog
• GeeksforGeeks: Design YouTube / Netflix